

Documento Técnico: Modelo de Datos — Proyecto HeartCoin

Introducción

Este documento describe de forma exhaustiva el modelo de datos del proyecto HeartCoin, implementado sobre PostgreSQL a través de Supabase. Se detalla cada entidad del esquema, sus relaciones, las reglas de integridad que las gobiernan, los patrones de diseño de datos adoptados de forma consistente a lo largo del proyecto, y los mecanismos —triggers, funciones y políticas de seguridad— que mantienen la consistencia del sistema sin intervención del cliente.

Objetivo

Documentar con precisión técnica la totalidad de las entidades del modelo de datos de HeartCoin, sus relaciones y reglas de integridad, con el fin de servir como referencia autorizada para el mantenimiento, la extensión y la auditoría del sistema.

Alcance

Este documento cubre las 20 tablas del esquema `public` que conforman el dominio funcional de la aplicación, la tabla `auth.users` de Supabase en tanto entidad raíz de identidad, y los mecanismos de consistencia (triggers, funciones `SECURITY DEFINER`) directamente asociados a la integridad de los datos. No cubre el detalle exhaustivo de cada política de Row Level Security (documentado en `ARQUITECTURA_TECNICA.md` y `ENDPOINTS.md`) ni aspectos de rendimiento o indexación no verificados directamente.

Metodología

La información aquí presentada se obtuvo mediante inspección directa del esquema en producción, consultando el catálogo de PostgreSQL (`information_schema`, `pg_constraint`, `pg_trigger`, `pg_proc`) a lo largo del desarrollo del proyecto, complementada con el análisis del código fuente de la aplicación cliente y las funciones de servidor que interactúan con cada entidad.

1. Modelo Conceptual: Dominios Funcionales

El modelo de datos se organiza, de forma implícita, en seis dominios funcionales interrelacionados:

1. **Identidad:** `auth.users` (Supabase) y sus tres extensiones de perfil por rol.
2. **Red social:** publicaciones, comentarios, likes, seguimiento y bloqueo entre usuarios.

3. **Participación cívica:** iniciativas, votos de apoyo, check-ins y proveedores.
4. **Campañas y votaciones:** votaciones de fundaciones, votos y comentarios asociados.
5. **Economía interna (HeartCoin):** el ledger de transacciones, único punto de convergencia de los dominios 3, 4 y 6.
6. **Marketplace empresarial:** beneficios y servicios ofrecidos por empresas, y sus respectivos canjes.

Una séptima entidad transversal, `saved_items`, permite a los usuarios marcar contenido de los dominios 3 y 4 como favorito, actuando como una relación de utilidad que no pertenece estrictamente a ningún dominio.

2. Entidad Raíz: `auth.users`

Gestionada íntegramente por Supabase Auth, esta tabla es el punto de origen de la identidad de todo usuario del sistema, sin importar su rol. **Treinta relaciones de clave foránea** en el esquema `public` apuntan a esta tabla, todas configuradas con `ON DELETE CASCADE` — característica que garantiza que la eliminación de una cuenta elimina, de forma automática y completa, la totalidad de los datos asociados en cualquier dominio, sin requerir lógica de limpieza manual.

3. Dominio: Identidad (Perfiles por Rol)

Tabla	Rol	Columnas relevantes	Particularidades
<code>personal_profiles</code>	Personal	<code>full_name</code> , <code>email</code> , <code>phone</code> , <code>country</code> , <code>city</code> , <code>bio</code> , <code>profile_type</code> , <code>avatar_url</code> , <code>hc_balance</code> , <code>notify_likes</code> , <code>notify_comments</code> , <code>notify_replies</code> , <code>is_public</code> , <code>original_auth_id</code> , <code>email_verified</code>	Única tabla que contiene <code>hc_balance</code> (derivado, no editable directo) y las columnas de integración con Original Auth
<code>organization_profiles</code>	Organización	<code>organization_name</code> , <code>representative_name</code> , <code>position</code> , <code>corporate_email</code> , <code>country</code> , <code>city</code> , <code>organization_type</code> , <code>impact_area</code> , <code>original_auth_id</code> , <code>email_verified</code>	Autora de iniciativas; su nombre se denormaliza hacia <code>iniciativas.organization_name</code>

company_profiles	Empresa	company_name, representative_name, position, corporate_email, country, city, industry, employee_range, main_goal, original_auth_id, email_verified	Autora de beneficios/servicios; su nombre se denormaliza hacia beneficios.company_name / servicios.company_name
------------------	---------	---	---

Las tres tablas comparten `id` como clave primaria y clave foránea directa hacia `auth.users(id)`, estableciendo una relación uno a uno entre la identidad de autenticación y el perfil funcional del usuario.

`personal_profiles.hc_balance` merece mención aparte: es un **valor derivado**, sincronizado automáticamente mediante trigger desde `hc_transactions` (dominio 5), y no debe interpretarse como un campo de escritura libre pese a no existir una restricción de columna que lo impida explícitamente a nivel de RLS.

4. Dominio: Red Social

Tabla	Propósito	Relación clave
publicaciones	Publicaciones del feed (texto, imagen, documento, ubicación)	<code>author_id</code> → <code>auth.users</code> ; opcionalmente referencia una iniciativa relacionada
post_likes	"Me gusta" sobre una publicación	Par (<code>post_id</code> , <code>user_id</code>)
comments	Comentarios sobre una publicación, con un nivel de respuestas	Auto-referencial vía <code>parent_comment_id</code> (ON DELETE CASCADE)
comment_likes	"Me gusta" sobre un comentario	Par (<code>comment_id</code> , <code>user_id</code>)
follows	Relación de seguimiento entre usuarios	Par (<code>follower_id</code> , <code>followed_id</code>)
blocked_users	Bloqueo de un usuario hacia otro	Par (<code>user_id</code> , <code>blocked_user_id</code>), con restricción CHECK que impide el auto-bloqueo

<code>notifications</code>	Notificaciones generadas por actividad social	<code>user_id</code> (destinatario); generada exclusivamente por triggers, nunca por el cliente
----------------------------	---	---

El campo `comments.parent_comment_id`, al estar configurado con `ON DELETE CASCADE` hacia la propia tabla `comments`, garantiza que eliminar un comentario raíz elimina automáticamente la totalidad de sus respuestas, sin requerir una operación de borrado recursivo explícita en la aplicación.

5. Dominio: Participación Cívica

Tabla	Propósito	Particularidades
<code>iniciativas</code>	Iniciativas de Voluntariado, Crowdfunding, Social o Ahorro	Estado (<code>borrador/votacion/activa</code>), método de verificación (<code>qr/manual/foto</code>), <code>hc_reward</code> configurable, <code>organization_name</code> denormalizado y sincronizado por trigger
<code>iniciativa_votes</code>	Apoyo de un usuario a una iniciativa	No otorga HC (decisión de diseño); al alcanzar <code>votes_goal</code> en estado <code>votacion</code> , dispara transición automática a <code>activa</code>
<code>iniciativa_checkins</code>	Check-in de asistencia a una iniciativa	Clave primaria compuesta (<code>iniciativa_id, user_id</code>); insertable únicamente vía la función <code>checkin_iniciativa()</code> ; dispara acreditación de HC y generación de certificado
<code>proveedores</code>	Catálogo de proveedores aliados para metas de ahorro	Sin política de escritura para el cliente; poblada exclusivamente por proceso administrativo

6. Dominio: Campañas y Votaciones

Tabla	Propósito	Particularidades
-------	-----------	------------------

votaciones	Campañas de fundaciones que compiten por votos	hc_reward y budget_items (JSONB con desglose porcentual de presupuesto)
votacion_votes	Voto de un usuario en una votación	Restricción única (votacion_id, user_id); dispara acreditación de HC
votacion_comments	Comentarios sobre una votación	Tabla independiente de comments, deliberadamente separada para no interferir con los triggers de notificaciones del dominio social

7. Dominio: Economía Interna (HeartCoin)

La tabla `hc_transactions` es la entidad de mayor relevancia arquitectónica del modelo de datos: constituye el **ledger contable** —fuente única de verdad— de todo movimiento de saldo en el sistema.

Columna	Tipo	Descripción
id	uuid	Identificador de la transacción
user_id	uuid	Usuario titular del movimiento (FK a auth.users, ON DELETE CASCADE)
amount	integer	Monto con signo (positivo = acreditación, negativo = cargo)
type	text	Restringido por CHECK a: voto, checkin, redemption_spend, redemption_cashback, ajuste
reference_type	text	Referencia polimórfica: iniciativa, votacion, beneficio, servicio, originalpay, originalpay_refund
reference_id	uuid	Identificador del registro de origen (nulo cuando no aplica, como en originalpay)

<code>external_reference</code>	<code>text</code>	Referencia de un sistema externo (ej. <code>order_number</code> de Original Pay); complementa a <code>reference_id</code> cuando el origen no es una entidad interna con UUID
<code>created_at</code>	<code>timestampz</code>	Fecha de la transacción

No existe una columna `hc_balance` editable directamente en ninguna tabla: el saldo mostrado en `personal_profiles.hc_balance` se deriva exclusivamente de la suma de esta tabla, sincronizada mediante el trigger `apply_hc_transaction` en cada inserción. No existe política de `INSERT` para el rol `authenticated` sobre `hc_transactions`: toda escritura proviene de funciones `SECURITY DEFINER` o de Edge Functions con clave de servicio.

8. Dominio: Marketplace Empresarial

Tabla	Propósito	Particularidades
<code>beneficios</code>	Ofertas de empresas canjeables con HC (descuento/cashback/beca/otro)	<code>company_name</code> denormalizado, estado, vencimiento, <code>max_redemptions</code>
<code>beneficio_redemptions</code>	Canje de un beneficio	Clave primaria compuesta (<code>beneficio_id</code> , <code>user_id</code>); insertable únicamente vía <code>redeem_beneficio()</code>
<code>servicios</code>	Catálogo de servicios empresariales (consultoría, diseño, etc.)	<code>pricing_type</code> (costo/cashback), independiente del catálogo de beneficios
<code>servicio_redemptions</code>	Canje de un servicio	Clave primaria compuesta (<code>servicio_id</code> , <code>user_id</code>); insertable únicamente vía <code>redeem_servicio()</code>

9. Entidad Transversal: `saved_items`

Permite a un usuario marcar como favorita una iniciativa o una votación, mediante los campos `item_type` (discriminador) e `item_id` (referencia genérica, sin clave foránea formal dado que apunta a dos tablas distintas según el tipo). Restricción única por (`user_id`, `item_type`, `item_id`).

10. Patrones de Diseño de Datos Observados

- **Patrón de ledger contable** (`hc_transactions`): en lugar de un campo de saldo mutable, el sistema registra cada movimiento de forma inmutable y deriva el saldo por sincronización automática. Este patrón es el de mayor solidez del modelo, al garantizar trazabilidad y auditabilidad completas.
- **Denormalización controlada con sincronización por trigger**: campos como `organization_name` (en `iniciativas`) y `company_name` (en `beneficios/servicios`) se duplican deliberadamente para evitar joins constantes en el cliente, manteniéndose sincronizados automáticamente ante cambios en la tabla de origen — nunca editables de forma independiente.
- **Referencia polimórfica limitada**: `hc_transactions.reference_type` + `reference_id/external_reference` permite que una sola tabla registre movimientos originados en múltiples dominios distintos, a costa de renunciar a la validación de integridad referencial automática de PostgreSQL para ese campo (mitigado por el hecho de que el campo únicamente se escribe desde un conjunto controlado y reducido de funciones de servidor).
- **Claves primarias compuestas para relaciones de "una vez por usuario"**: `iniciativa_checkins`, `beneficio_redemptions`, `servicio_redemptions` utilizan pares de columnas como clave primaria en lugar de un `id` autogenerado, expresando directamente en el esquema la regla de negocio "un check-in/canje por usuario por entidad".
- **Cascada de eliminación uniforme**: la totalidad de las relaciones hacia `auth.users` utiliza `ON DELETE CASCADE`, permitiendo que la eliminación de cuenta sea una operación atómica y completa sin lógica adicional distribuida por la aplicación.

11. Mecanismos de Consistencia Automática (Triggers)

Trigger	Tabla origen	Efecto
<code>apply_hc_transaction</code>	<code>hc_transactions</code>	Sincroniza <code>personal_profiles.hc_balance</code> en cada inserción
<code>sync_post_comments_count / sync_post_comments_count_on_delete</code>	<code>comments</code>	Mantiene <code>publicaciones.comments_count</code> sincronizado en inserción y borrado
Triggers de notificación (<code>notify_post_like</code> , <code>notify_new_comment</code> , <code>notify_comment_like</code>)	<code>post_likes</code> , <code>comments</code> , <code>comment_likes</code>	Generan notificaciones respetando las preferencias de opt-out del destinatario

<code>grant_hc_for_checkin</code>	<code>iniciativa_checkins</code>	Acredita <code>iniciativas.hc_reward</code> al usuario
<code>grant_hc_for_votacion_vote</code>	<code>votacion_votes</code>	Acredita <code>votaciones.hc_reward</code> al usuario
Trigger de sincronización de <code>organization_name</code>	<code>iniciativas</code>	Sincroniza desde <code>organization_profiles</code> según <code>author_id</code>

Este conjunto de triggers materializa el principio de que **ninguna cifra derivada del sistema se calcula ni se escribe desde el cliente**, reduciendo la superficie de manipulación indebida y garantizando consistencia incluso ante clientes no oficiales.

Problemas Detectados

1. **La referencia polimórfica de `hc_transactions` carece de restricción `CHECK` sobre `reference_type`**, a diferencia del campo `type`, que sí está formalmente restringido. Su validez depende únicamente de que las funciones de escritura sean disciplinadas, no de una garantía del esquema.
2. **Ausencia de un trigger `AFTER DELETE` genérico para el ledger de HC**: se identificó, durante pruebas de la integración con Original Pay, que borrar filas de `hc_transactions` directamente (operación administrativa, no de aplicación) no revierte automáticamente `hc_balance`, dado que el trigger existente reacciona únicamente a inserciones. Esto es coherente con el diseño (el ledger no está pensado para borrarse), pero representa un riesgo si se realizan correcciones manuales sin el cuidado correspondiente.
3. **Ausencia de restricción de unicidad sobre `external_reference` en `hc_transactions`**: la idempotencia ante reintentos de integraciones externas (como el webhook de Original Pay) se resuelve actualmente mediante lógica de aplicación (verificación previa a la inserción), no mediante una restricción de base de datos que lo garantice de forma absoluta ante escritura concurrente.

Riesgos e Impactos

- El riesgo asociado al punto 2 es de **impacto operativo, no de seguridad**: únicamente materializable mediante intervención administrativa directa sobre la base de datos, no accesible desde la aplicación cliente ni desde las integraciones externas.
- El riesgo asociado al punto 3 es de **impacto financiero-operativo bajo**, dado que la lógica de aplicación ya implementada (verificación de existencia previa a la inserción) mitiga el caso común de reintento de webhook, aunque no ofrece la garantía absoluta que proporcionaría una restricción de unicidad a nivel de base de datos ante escritura verdaderamente concurrente.

Fortalezas Identificadas

- Modelo de datos altamente normalizado en sus relaciones centrales, con denormalización aplicada de forma selectiva y siempre sincronizada automáticamente.
- Ledger contable como patrón central de la economía interna, superior en auditabilidad a un campo de saldo mutable simple.
- Integridad referencial consistente y verificada (30 relaciones hacia `auth.users`, la totalidad con `ON DELETE CASCADE`).
- Expresión de reglas de negocio directamente en el esquema mediante claves primarias compuestas, reduciendo la posibilidad de duplicidad de datos por error de aplicación.

Recomendaciones

1. Incorporar una restricción `CHECK` sobre `hc_transactions.reference_type`, listando explícitamente los valores válidos actualmente en uso (`iniciativa`, `votacion`, `beneficio`, `servicio`, `originalpay`, `originalpay_refund`), replicando el mismo nivel de garantía ya existente sobre la columna `type`.
2. Evaluar la incorporación de una restricción de unicidad parcial sobre (`reference_type`, `external_reference`) cuando corresponda, para elevar la garantía de idempotencia de nivel aplicación a nivel de base de datos.
3. Documentar explícitamente, como procedimiento operativo, que cualquier corrección manual sobre `hc_transactions` debe acompañarse de una transacción compensatoria (inserción), nunca de un borrado directo, dado que solo las inserciones disparan la sincronización del saldo.

Conclusión

El modelo de datos de HeartCoin refleja un diseño maduro para el dominio que atiende, particularmente evidenciado en el tratamiento de la economía interna de HeartCoin mediante un patrón de ledger contable auditable, y en la integridad referencial consistente que sostiene el ciclo de vida completo de una cuenta de usuario. Las áreas de mejora identificadas son de naturaleza incremental —fortalecimiento de restricciones ya parcialmente implementadas— y no representan debilidades estructurales del diseño general.

Anexo: Diccionario de Entidades (Resumen)

Dominio	Entidades
Identidad	<code>personal_profiles</code> , <code>organization_profiles</code> , <code>company_profiles</code>

Red social	publicaciones, post_likes, comments, comment_likes, follows, blocked_users, notifications
Participación cívica	iniciativas, iniciativa_votes, iniciativa_checkins, proveedores
Campañas y votaciones	votaciones, votacion_votes, votacion_comments
Economía interna	hc_transactions
Marketplace empresarial	beneficios, beneficio_redemptions, servicios, servicio_redemptions
Transversal	saved_items